CS235 COA

Dr.K.Sridhar Patnaik
Associate Professor
Dept of CSE,BIT Mesra
kspatnaik@bitmesra.ac.in

Course Objectives

- To Learn:

1)How computers work,basic principles.

2)How to analyze their performance(or how not to)

3)How computers are designed and built.

4)Issues affecting modern processors(cache,pipelines etc)

Course Outcomes:

Course Outcomes:

CO1Describe the merits and pitfalls in computer performance measurements and analyse the impact of instruction set architecture on cost-performance of computer design.

CO2Explain Digital Logic Circuits ,Data Representation, Register and Processor level Design and Instruction Set architecture

CO3Solve problems related to computer arithmetic and Determine which hardware blocks and control lines are used for specific instructions

CO4Design a pipeline for consistent execution of instructions with minimum hazards.

CO5Explain memory organization ,I/O organization and its impact on computer cost/performance.

Computer Organization & Design:The Hardware/Software Interface
- David Patterson and John Hennessey. 1-55860-604-1. HB Latest Morgan Kaufman

Course Motivation

This knowledge will be useful if you need to

1)Design/built a new computer-*rare opportunity*

2)Design/built a new version of a computer.

3)Improve software performance.

4)Purchase a computer

5)Provide a solution with embedded computer.

Introduction

- I think it’s fair to say that personal computers have become the most empowering tool we’ve ever created. They’re tools of communication, they’re tools of creativity, and they can be shaped by their user.

  Bill Gates, February 24, 2004

Computers?

A computer is a general purpose device that can be programmed to process information, and yield meaningful results.

Computers for..

- Processing and Communication
- For processing (ex. Numbers) requires processor , memory and I/O(input-output)

Processor-for processing.

Memory-for storage.

I/O -can be considered as extended memory.(Also helps us in interacting with m/c)

Computer Architecture Vs Computer Organization

You can study Computer Systems from users(car driver) point of view or designer's(car mechanic) point of view

Computer Architecture :The view of a computer as presented to software designers.

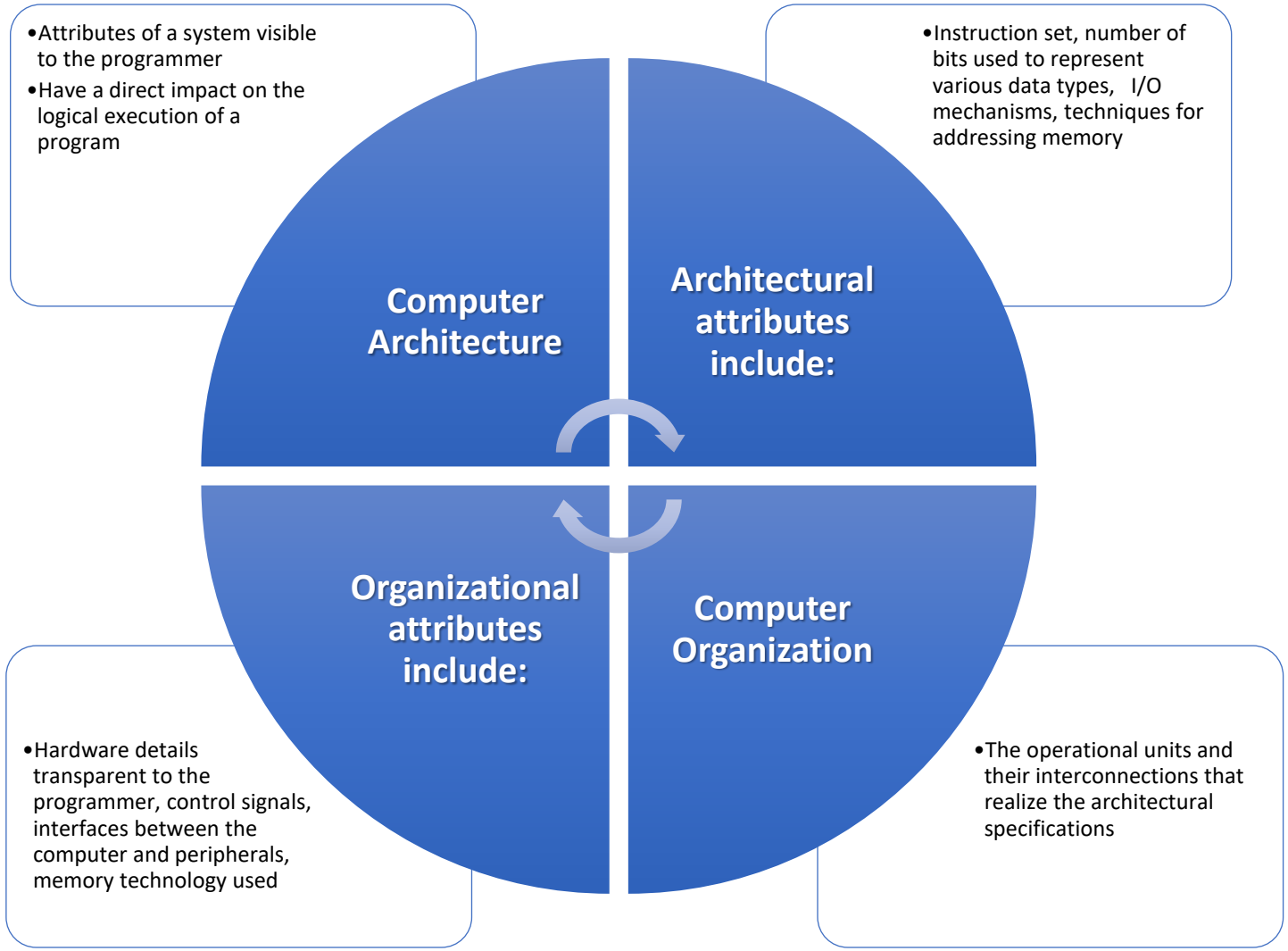
i.e. from designers(hardware) point of view.

Building Architecture: Structural Design(Civil Engg)

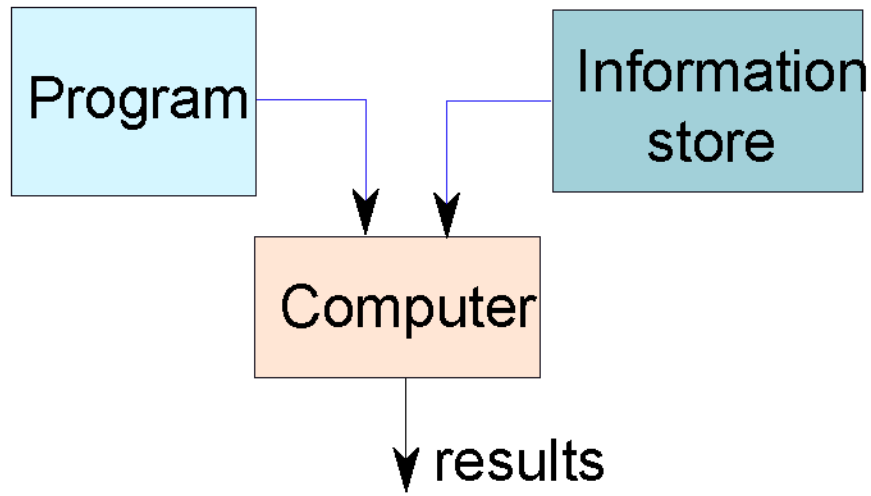
Computer Architecture: Circuit Design(EE)

Computer Organization: The actual implementation of a computer in hardware.

i.e. from users(programmers/software) point of view.


•Attributes of a system visible to the programmer
•Have a direct impact on the logical execution of a program
Computer Architecture
Architectural attributes include:
•Instruction set, number of bits used to represent various data types, I/O mechanisms, techniques for addressing memory
Organizational attributes include:
Computer Organization
•Hardware details transparent to the programmer, control signals, interfaces between the computer and peripherals, memory technology used
•The operational units and their interconnections that realize the architectural specifications

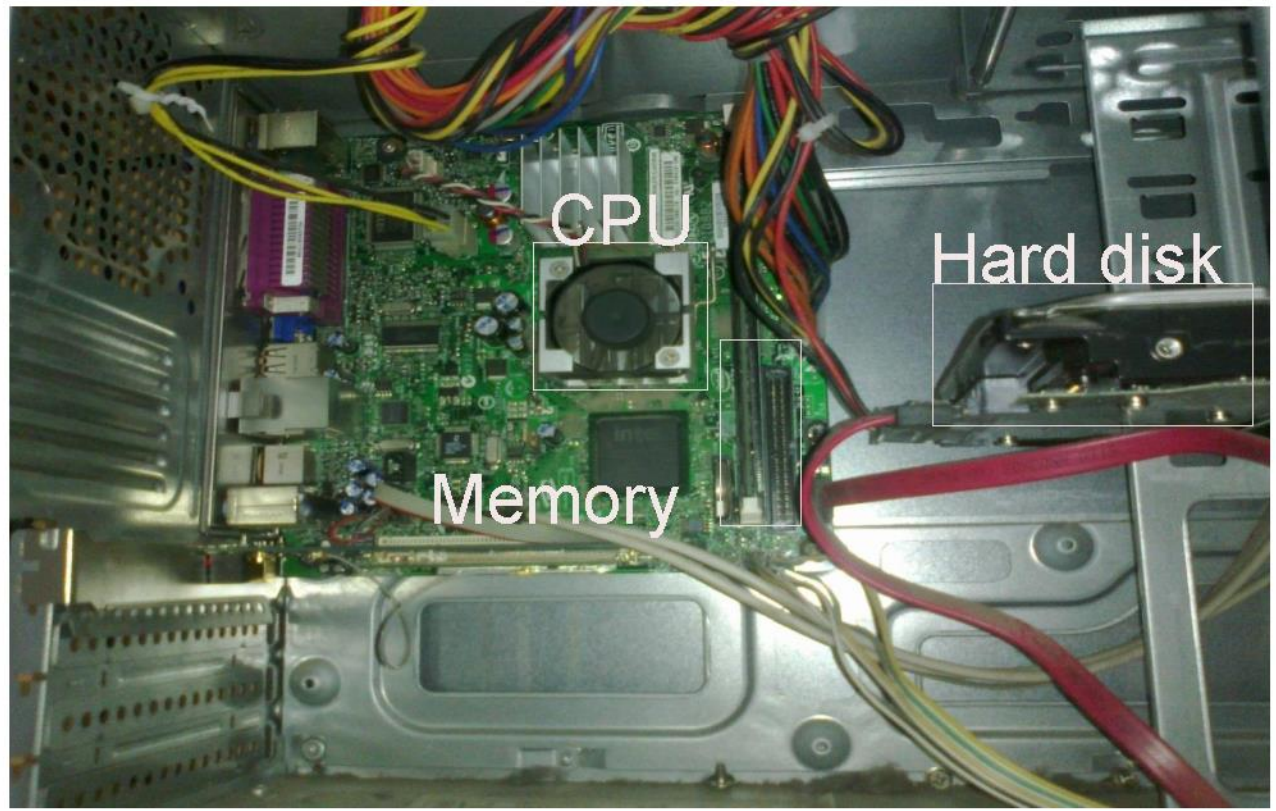
- Ex.Mutiplier,from organization point of view you know there is a multiplier, one need not bother about how it is designed .Similarly there is an instruction set and the user know the instruction that is enough and he is not bothered about how it is implemented.
- So how multiplier and instruction set are implemented is the job of the designers.
- Application spectrum



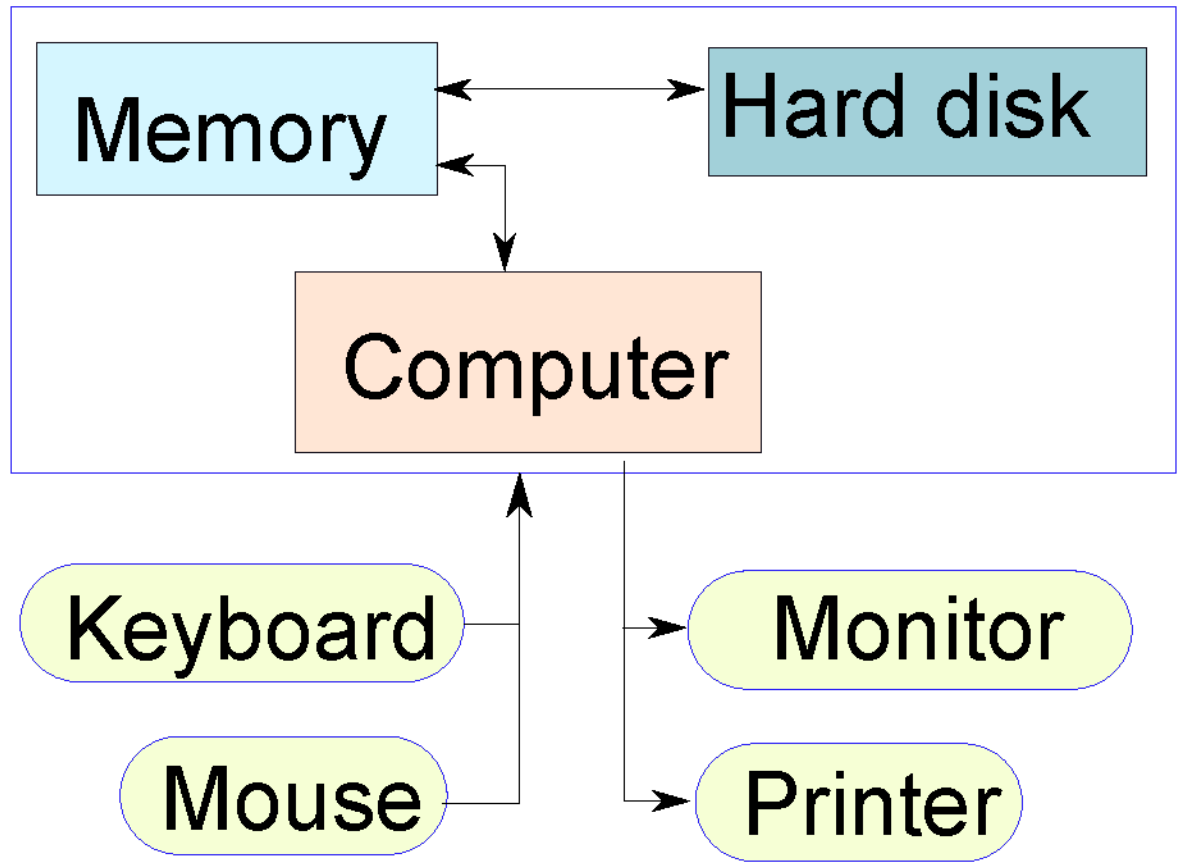
- Program – List of instructions given to the computer
- Information store – data, images, files, videos
- Computer – Process the information store according to the instructions in the program

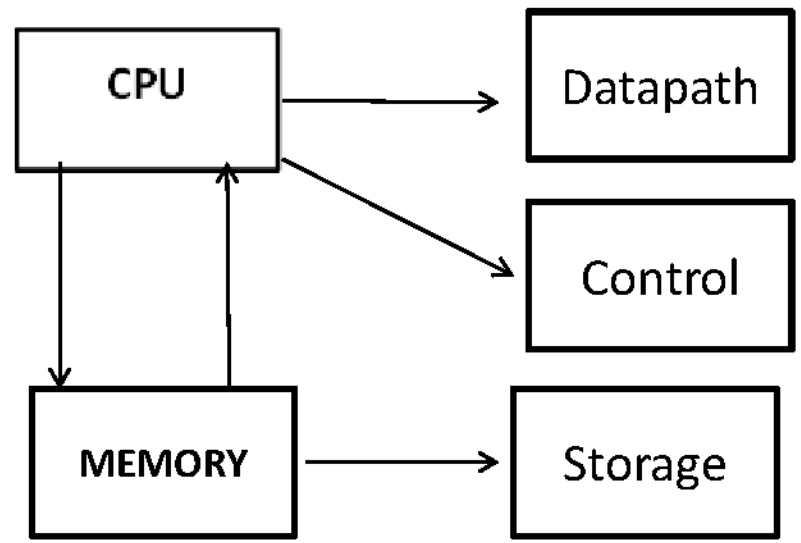
Inside

* Let us take the lid off a desktop computer



Let us make it a full system …



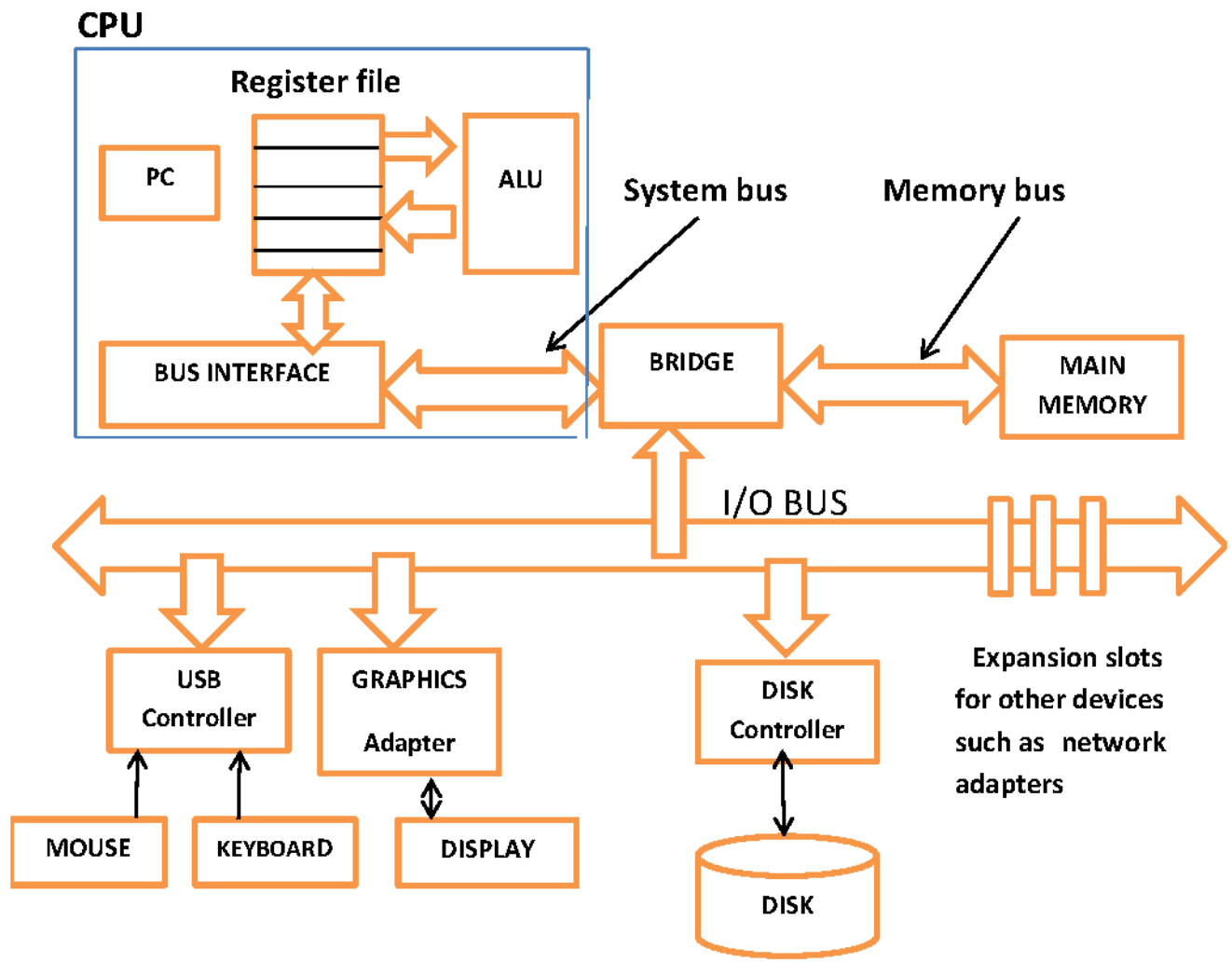

CPU
Datapath
Control
MEMORY
Storage

- Processor keep addressing the memory and get the data from memory for processing.
- Processor consists(DP ckts+CP ckts)
- Processor must be able to set some kind of path to deal or handle the data for processing (data path setting).
- Also able to carry out the processing through a sequence of control signals.
- The one which interact the CPU indirectly is the storage(magnetic tape, disc system etc)
- The CPU deals with memory in the same way as it deals with I/O.

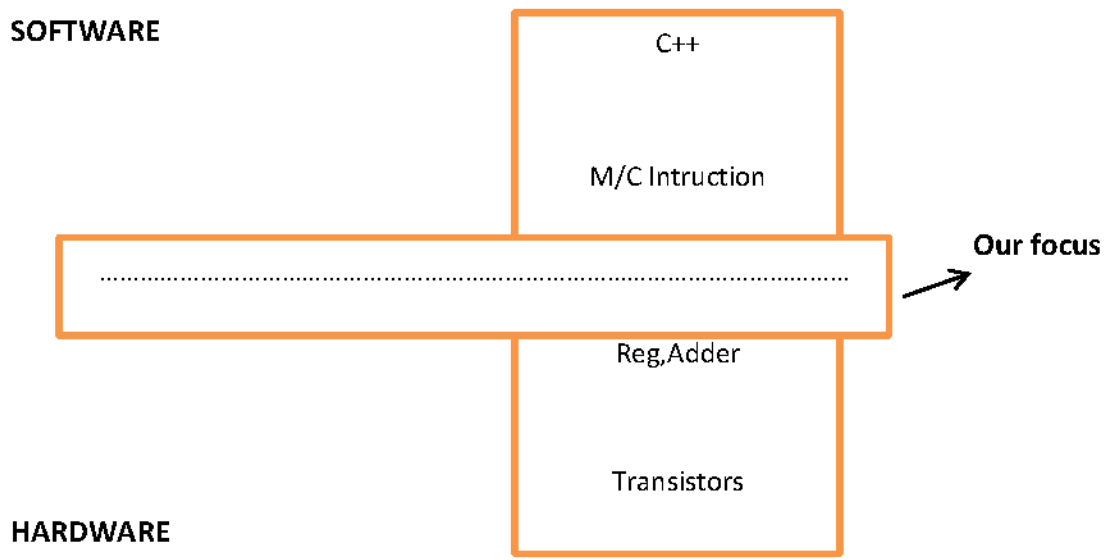
SOFTWARE ABSTRACTION

- int sum(int x,int y) HLL(C)
- {
- int z=x+y;
- return z;}
- 0x401040<sum>:0x55 MACHINE CODE
- 0x89
- 0xe5
- 0x8b
- 0x45
- 0x0C
- 0x03
- 0x45
- 0x08
- 0x89
- 0xec
- 0x5d
- 0xc3
- _sum: ASSEMBLY
- pushl%ebp
- movl%esp,%ebp
- movl 12(%ebp),%eax
- addl 8(%ebp),%eax
- movl %ebp,%esp
- popl %ebp
- ret

HARDWARE ABSTRACTION



HARDWARE/SOFTWARE INTERFACE



How does an Electronic Computer Differ from our Brain ?

Feature	Computer	Our Brilliant Brain
Intelligence	Dumb	Intelligent
Speed of basic calculations	Ultra-fast	Slow
Can get tired	Never	After sometime
Can get bored	Never	Almost always

- Computers are ultra-fast and ultra-dumb

What Can a Computer Understand ?

* Computer can clearly NOT understand instructions of the form
  * Multiply two matrices
  * Compute the determinant of a matrix
  * Find the shortest path between Mumbai and Delhi
* They understand :
  * Add *a* + *b* to get *c*
  * Multiply *a* + *b* to get *c*

Architecture levels

- Instruction set architecture:

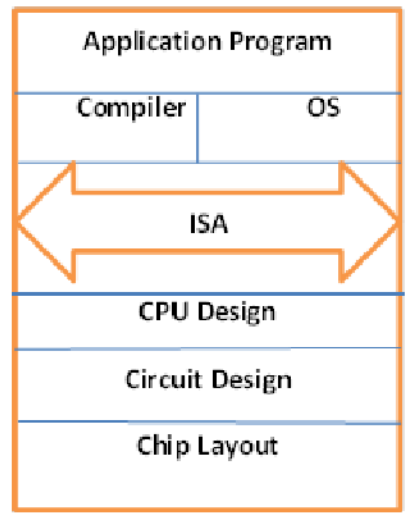
Lowest level visible to programmer

- Micro architecture:

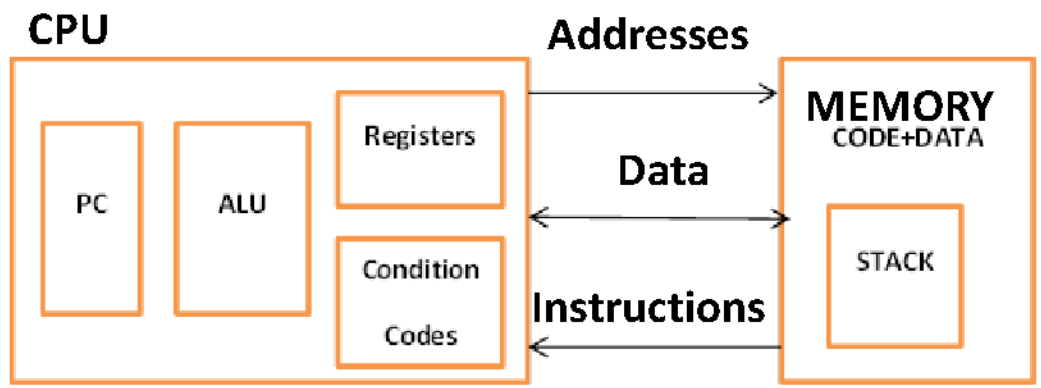
Fills the gap between instructions and logic modules

The semantics of all the instructions supported by a processor is known as its instruction set architecture (ISA). This includes the semantics of the instructions themselves, along with their operands, and interfaces with peripheral devices.

Instruction Set Architecture



Abstract Machine



- Programmer -Visible State:

PC-Program Counter

Register File-Heavily used data

Condition Codes

Memory-Byte Array

-Code+Data

-Stack

Why different processors?

- What is the difference between processors used in desk-tops, lap-tops,mobile phones, washing machines etc.?
- Performance/speed
- Power consumption
- Cost
- General purpose/special purpose

Topics to be Covered

- Performance issues
- A specific instruction set architecture
- Arithmetic and how to build an ALU
- Constructing a processor to execute instruction
- Pipelining to improve performance
- Memory:Caches and Virtual memory
- Input/output

Features of an ISA

* Example of instructions in an ISA
  * Arithmetic instructions : add, sub, mul, div
  * Logical instructions : and, or, not
  * Data transfer/movement instructions
* Complete
  * It should be able to implement all the programs that users may write.

Features of an ISA – II

* Concise
  * The instruction set should have a limited size. Typically an ISA contains 32-1000 instructions.
* Generic
  * Instructions should not be too specialized, e.g. add14 (adds a number with 14) instruction is too specialized
* Simple
  * Should not be very complicated.

Designing an ISA

* Important questions that need to be answered :
  * How many instructions should we have ?
  * What should they do ?
  * How complicated should they be ?

Two different paradigms : RISC and CISC

RISC
(Reduced Instruction Set Computer)

CISC
(Complex Instruction Set Computer)

RISC vs CISC

A reduced instruction set computer (RISC) implements simple instructions that have a simple and regular structure. The number of instructions is typically a small number (64 to 128). Examples: ARM, IBM PowerPC, HP PA-RISC

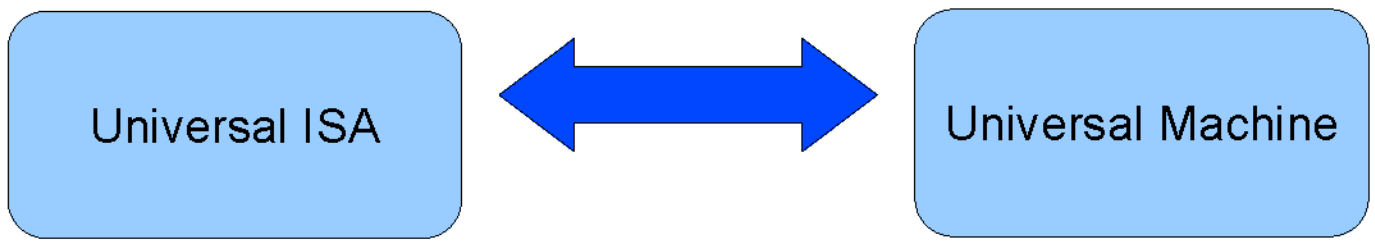
A complex instruction set computer (CISC) implements complex instructions that are highly irregular, take multiple operands, and implement complex functionalities. Secondly, the number of instructions is large (typically 500+). Examples: Intel x86, VAX

Completeness of an ISA – II

How to ensure that we have just enough instructions such that we can implement every possible program that we might want to write ?

Answer:

* Let us look at results in theoretical computer science
* Is there an universal ISA ?

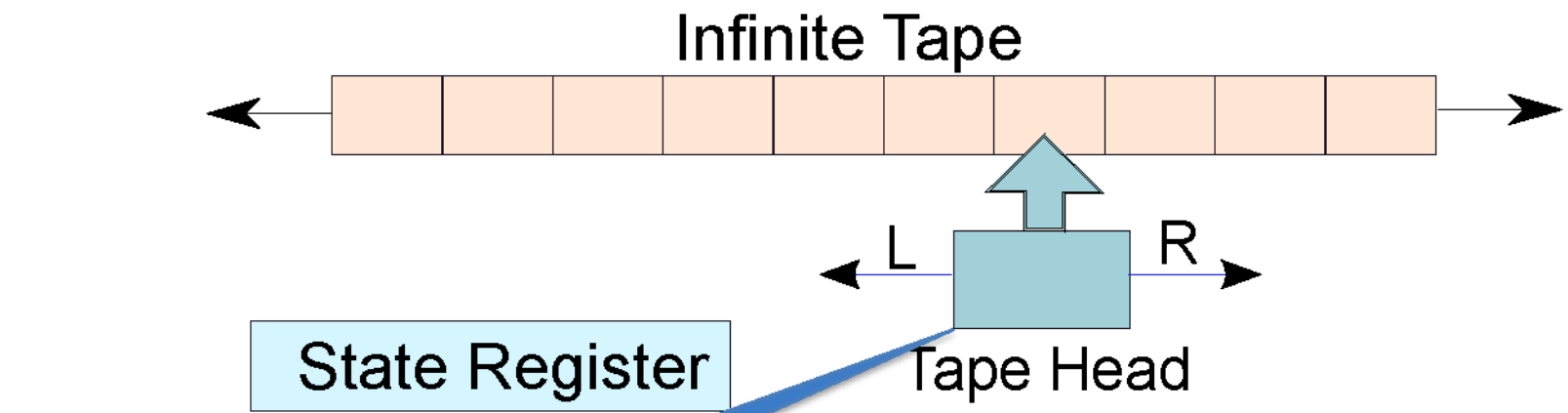


The universal machine has a set of basic actions, and each such **action** can be interpreted as an instruction.

The Turing Machine – Alan Turing

* Facts about Alan Turing
  * Known as the father of computer science
  * Discovered the Turing machine that is the most powerful computing device known to man
  * Indian connection : His father worked with the Indian Civil Service at the time he was born. He was posted in Chhatrapur, Odisha.

Turing Machine



- The tape head can only move left or right

Action Table

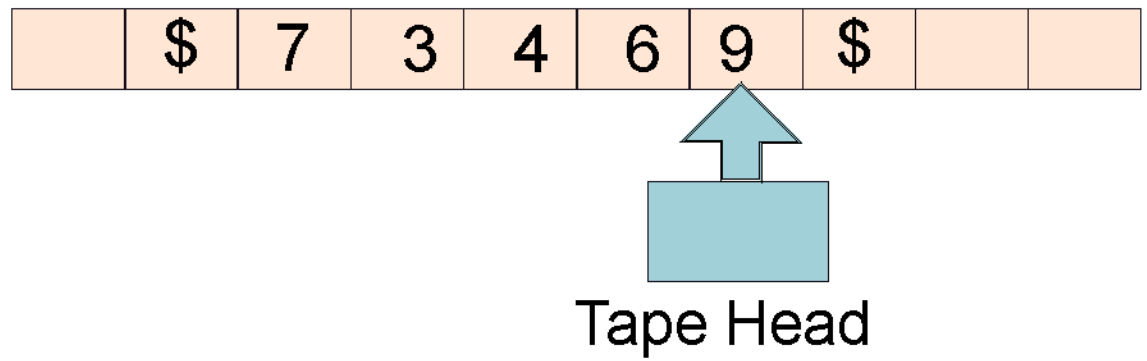
(old state, old symbol) -> (new state, new symbol, left/right)

Operation of a Turing Machine

* There is an inifinite tape that extends to the left and right. It consists of an infinite number of cells.
* The tape head points to a cell, and can either move 1 cell to the left or right
* Based on the symbol in the cell, and its current state, the Turing machine computes the transition :
  * Computes the next state
  * Overwrites the symbol in the cell (or keeps it the same)
  * Moves to the left or right by 1 cell
* The action table records the rules for the transitions.

Example of a Turing Machine

- *Design a Turing machine to increment a number by 1.*



* Start from the rightmost position. (state = 1)
* If (state = 1), replace a number x, by x+1 mod 10
* The new state is equal to the value of the carry
* Keep going left till the '$' sign

More about the Turing Machine

* This machine is extremely simple, and extremely powerful
    * We can solve all kinds of problems – mathematical problems, engineering analyses, protein folding, computer games, …
    * Try to use the Turing machine to solve many more types of problems (TO DO)

Church-Turing Thesis

Church-Turing thesis: Any real-world computation can be translated into an equivalent computation involving a Turing machine. (source: Wolfram Mathworld).

Any computing system that is equivalent to a Turing machine is said to be Turing complete.

Universal Turing Machine

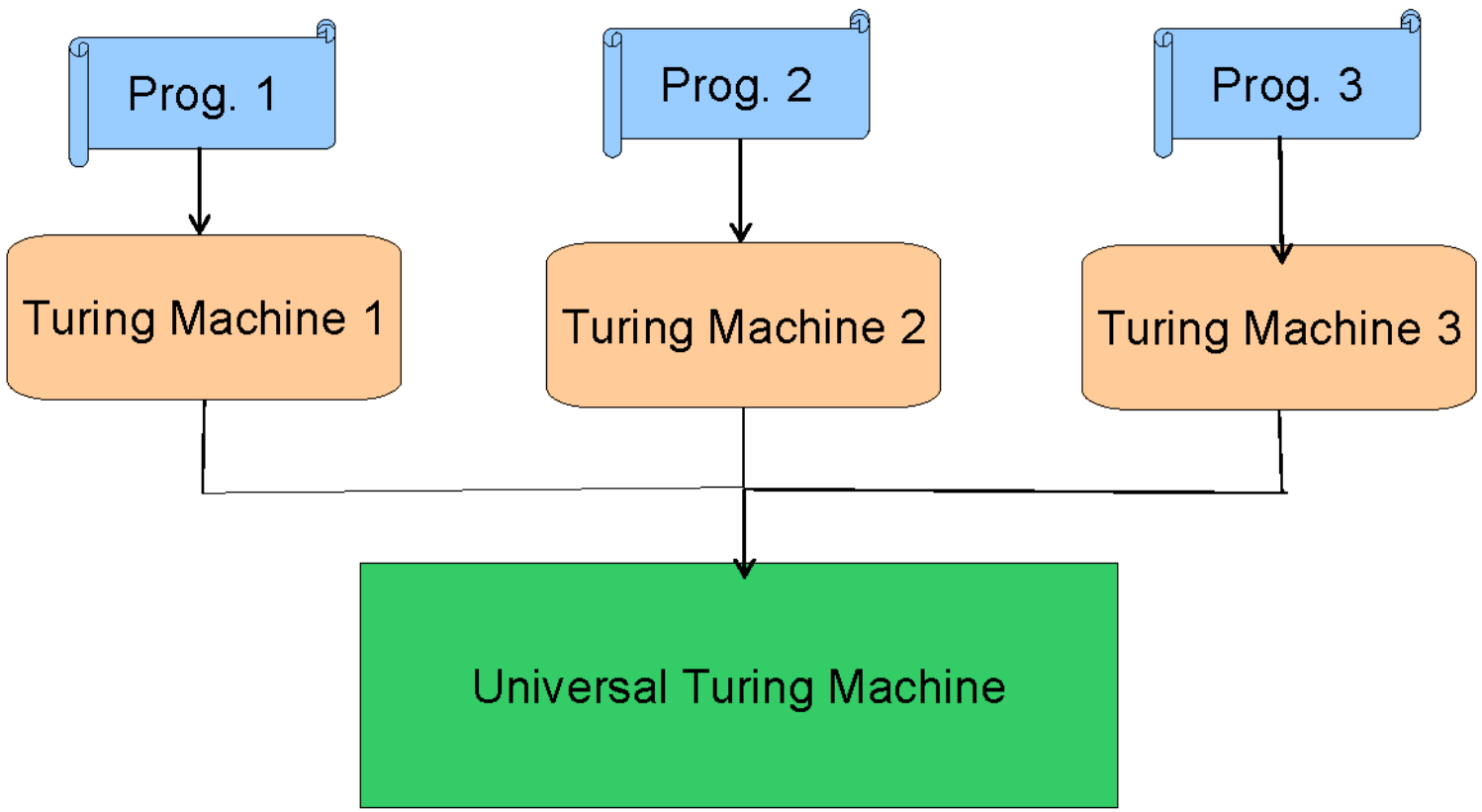
*For every problem in the world, we can design a Turing Machine (Church-Turing thesis)

*Can we design a universal Turing machine that can simulate any Turing machine. This will make it a universal machine (UTM)

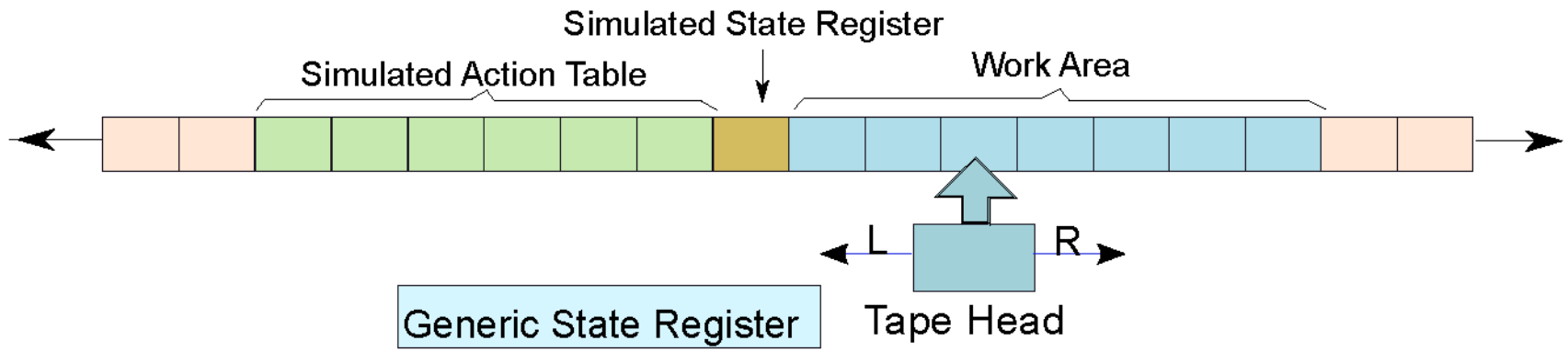
*Why not? The logic of a Turing machine is really simple. We need to move the tape head left, or right, and update the symbol and state based on the action table. A UTM can easily do this.

*A UTM needs to have an action table, state register, and tape that can simulate any arbitrary Turing machine.

Universal Turing Machine

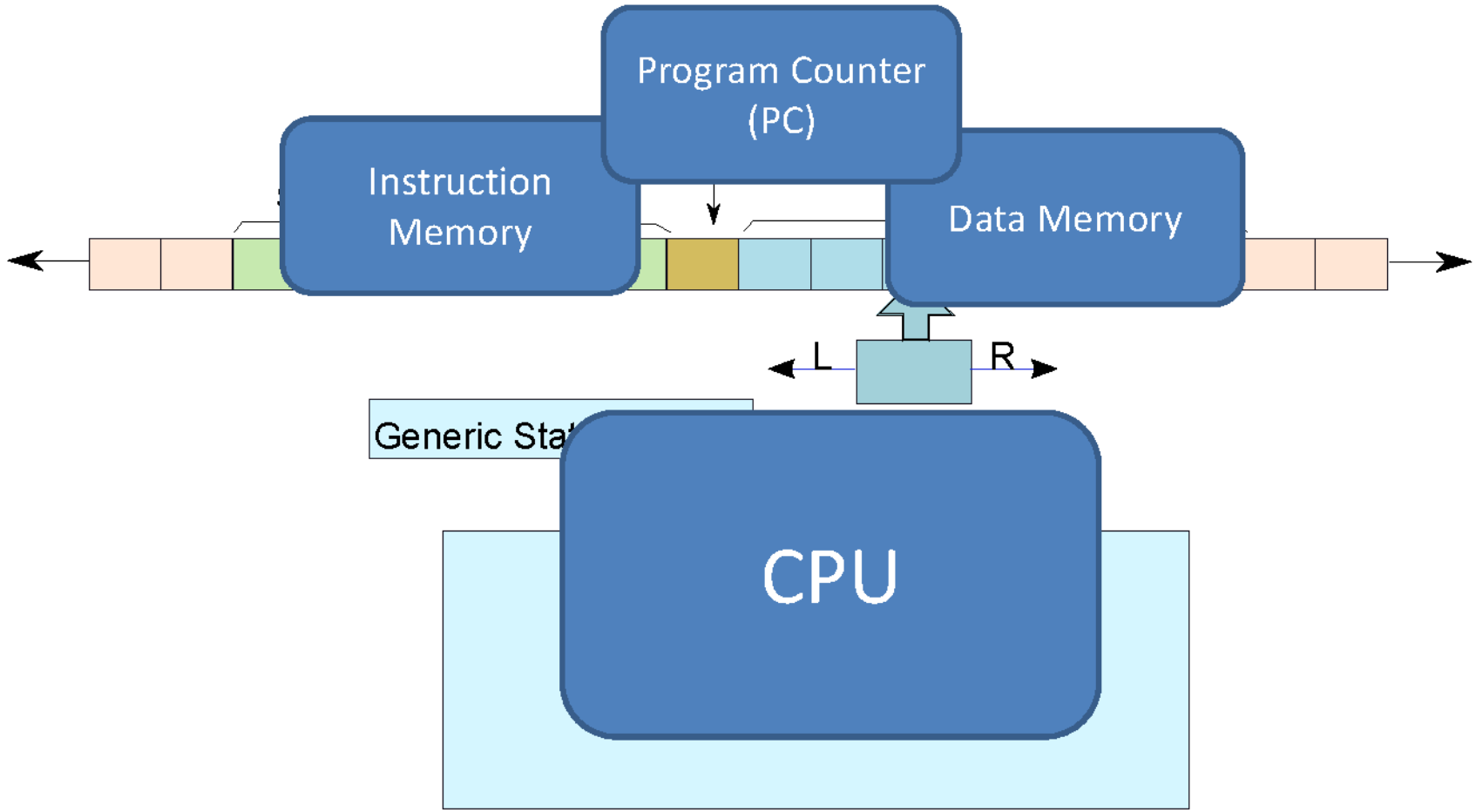


A Universal Turing Machine

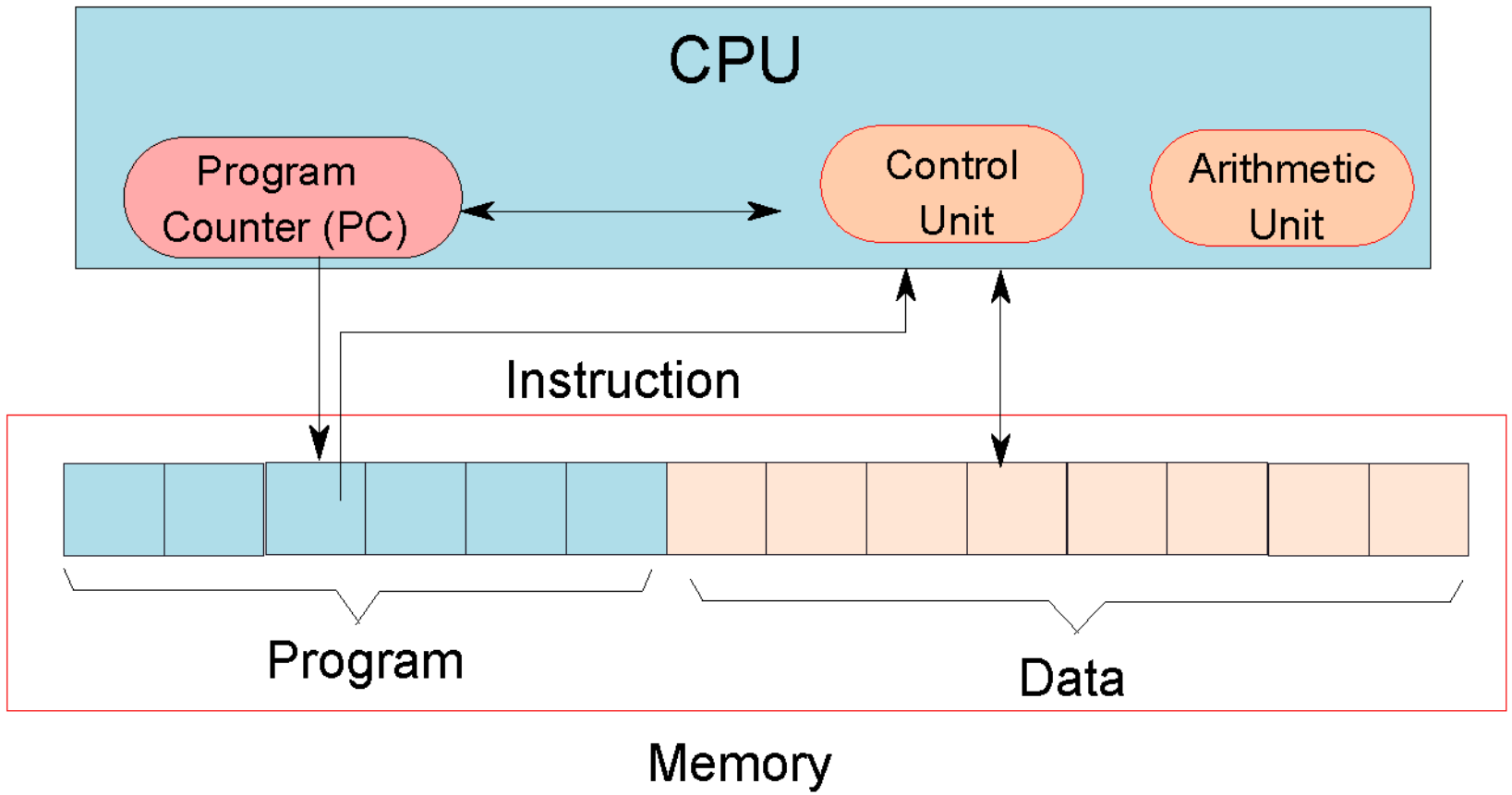


Generic Action Table

A Universal Turing Machine - II



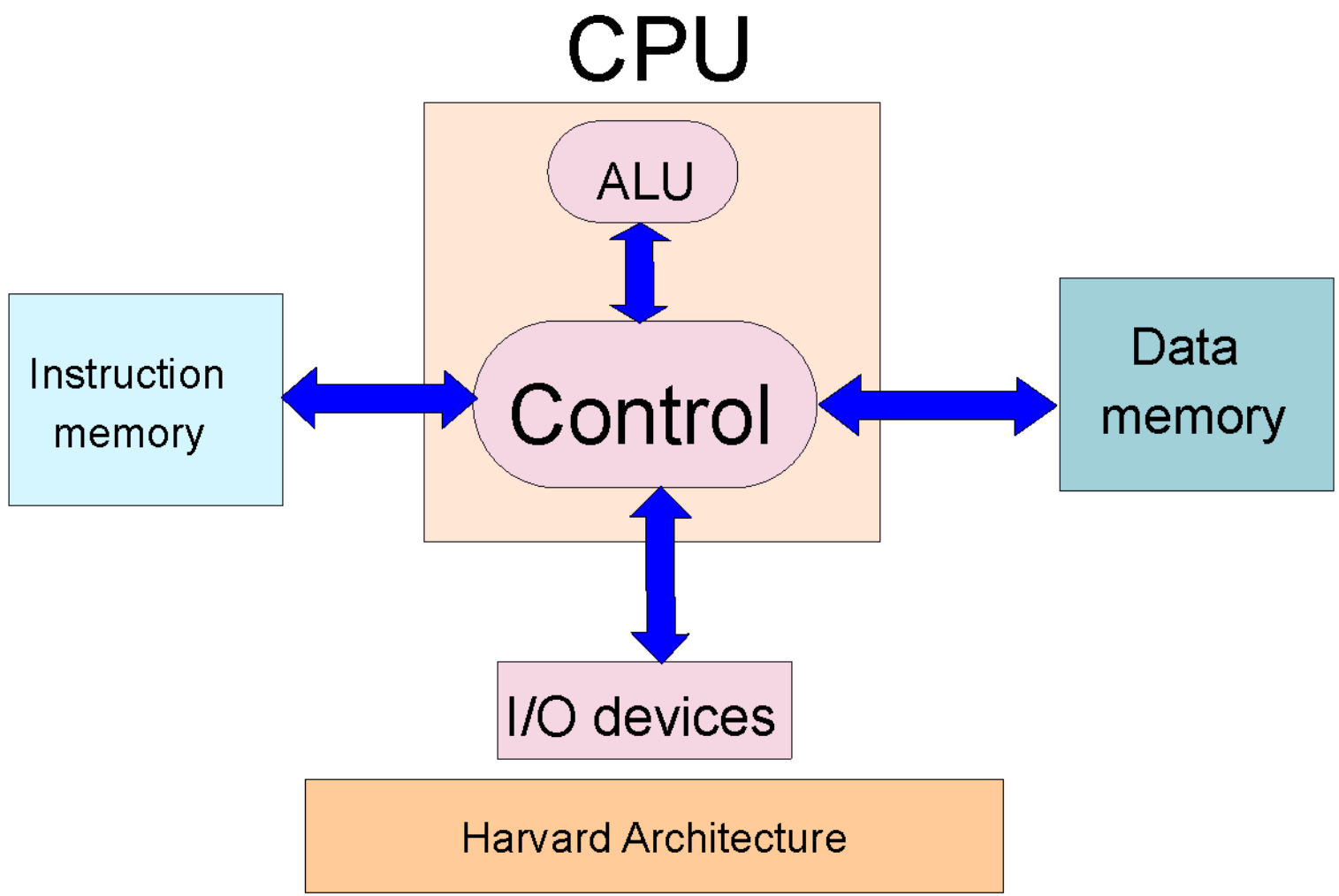
Computer Inspired from the Turing Machine



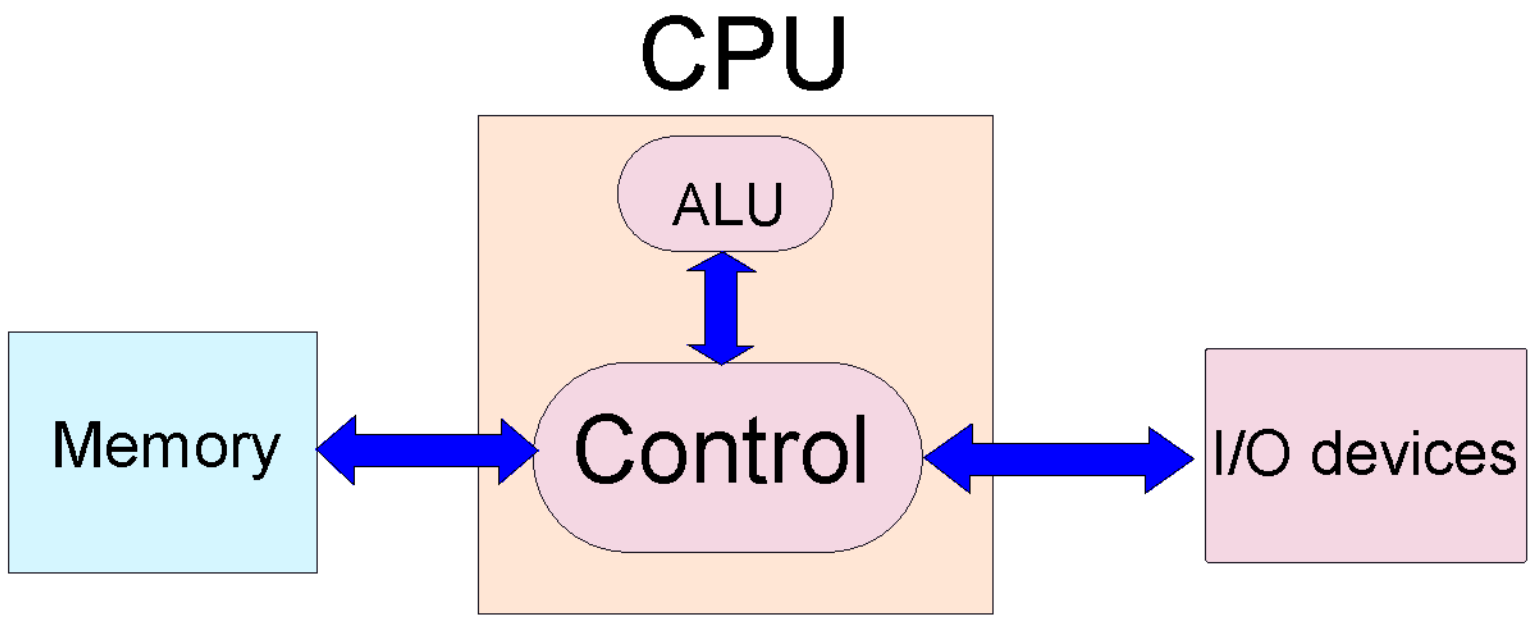
Elements of a Computer

* Memory (array of bytes) contains
  * The program, which is a sequence of instructions
  * The program data → variables, and constants
* The program counter(PC) points to an instruction in a program
  * After executing an instruction, it points to the next instruction by default
  * A branch instruction makes the PC point to another instruction (not in sequence)
* CPU (Central Processing Unit) contains the
  * Program counter, instruction execution units

Designing Practical Machines



Von-Neumann Architecture



Problems with Harvard/ Von-Neumann Architectures

* The memory is assumed to be one large array of bytes
  * It is very very slow

General Rule: Larger is a structure, slower it is

  * Solution:
    * Have a small array of named locations **(registers)** that can be used by instructions
    * This small array is very fast

Insight: Accesses exhibit locality (tend to use the same variables frequently in the same window of time)

Uses of Registers

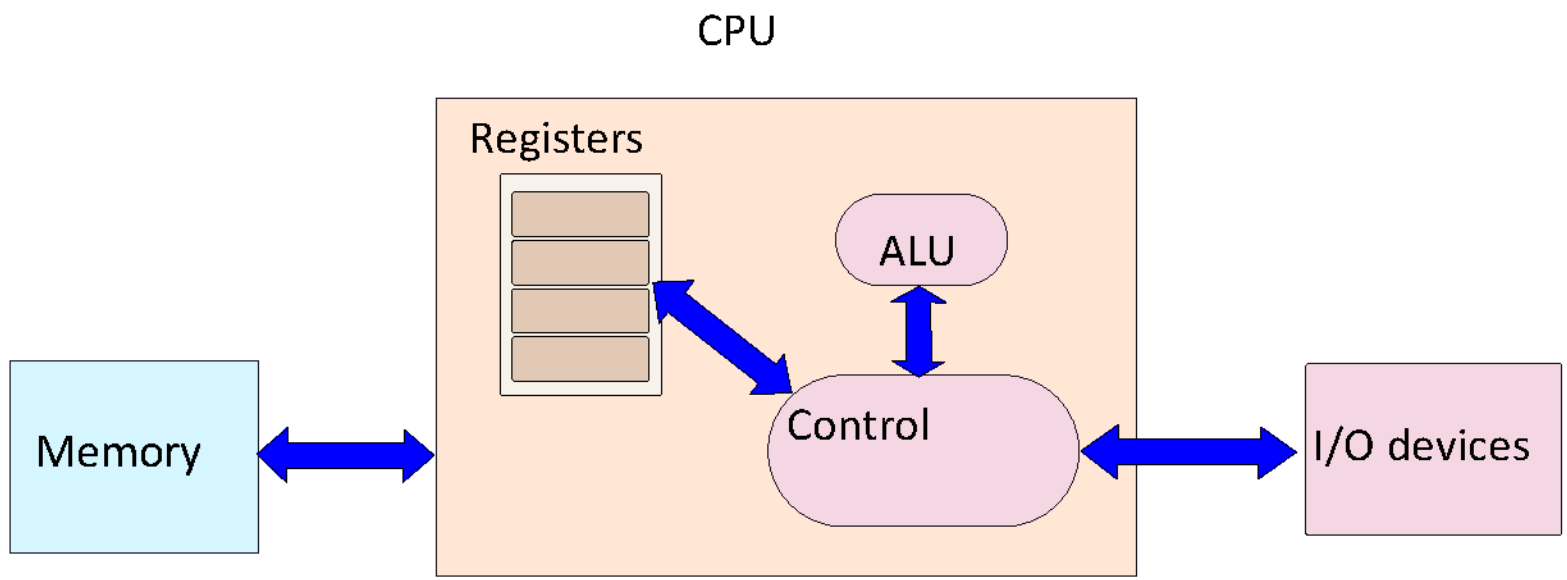
* A CPU (Processor) contains set of registers (16-64)
* These are named storage locations.
* Typically values are loaded from memory to registers.
* Arithmetic/logical instructions use registers as input operands
* Finally, data is stored back into their memory locations.

Example of a Program in Machine Language with Registers

```
1: r1 = mem[b] // load b
2: r2 = mem[c] // load c
3: r3 = r1 + r2 // add b and c
4: mem[a] = r3 // save the result
```

* r1, r2, and r3, are registers
* mem → array of bytes representing memory

Machine with Registers



Performance Measure

- When we say one computer has better performance than another , what do we mean?

Airplane	Passenger Capacity	Cruising Range(miles)	Cruising speed(mph)	Passenger throughput (passenger X mph)
A1	300	4000	600	180000
A2	400	3500	600	240000
A3	130	3400	1000	130000
A4	140	8000	500	70000

- Consider different measures of performance, the plane with highest cruising speed is A3 ,the plane with the longest range is A4,and the plane with the largest capacity is the A2 .

- Suppose we define performance in terms of speed.Then the fastest plane as the one with the highest cruising speed,taking less passengers from one point to another in the least time.

- If you are interested in transporting 400 passengers than A2 would clearly be the fastest.

- Similarly , we can define computer performance in several different ways

- If you are running a program on two desktop computers, The faster one is the one that gets the job done first.
- If you were running a datacenter that had several servers running jobs submitted by many users,the faster computer was the one that completed the most jobs during a day.
- As an individual computer user we are interested in reducing **response time(execution time).**
- Datacenter managers are often interested in increasing **throughput or bandwidth(total amount of work done in a given time)**
- **Hence in most cases we need different performance metrics as well as different sets of applications to benchmark embedded and desktop computers,which are more focused on response time,versus servers,which are more focused on throughput**

Throughput and Response time

- Do the following changes to a computer system increase throughput, decrease response time:

1.Replacing the processor in a computer with a faster version.

2.Adding additional processors to a system that uses multiple processors for separate task-ex searching the www.

In case 1 decreasing response time almost always improves throughput, hence case 1 improves both.

In case 2 throughput only increases.

Note-The demand for processing in the second case was almost as large as the throughput,the system might force requests to queue up.

In this case the throughput could also improve response time, since it would reduce the waiting time in the queue.

Thus, in many real computer systems, changing either execution time or throughput often affects the other.

Performance Measure Equations

- Performance=1/Execution time
- For two computers X and Y,if the performance of X is greater than Y ,
- $P_X > P_Y = \frac{1}{Exe.time_X} > \frac{1}{Exe.time_Y} = Exe.time_Y > Exe.timeX$
- Do this-If computer A runs a program in 10 sec and computer B runs the same program in 15 sec,how much faster is A than B?
- Response time,or elapsed time-Total time to complete a task,including disk accesses,memory accesses,I/O activities,OS overhead
- CPU Execution time/CPU time-The actual time the CPU spends computing specific task.(does not include time spent waiting for I/O or running other programs.
- CPU time=user CPU time+system CPU time

User CPU time(CPU time spent in the the program)

System CPU time(CPU time spent in the OS performing tasks on behalf of the program)

- Differentiating user and system CPU time is difficult to do accurately ,because it is hard to assign responsibility for OS activities to one user program rather than another and because of the functionality differences among OSs.
- **CPU execution time for a program**=CPU clock cycles for a program X Clock cycle time.

=CPU clock cycles for a program /Clock rate

Note-The hardware designer can improve performance by reducing the number of clock cycles required for a program or the length of the clock cycle.

The designer often faces a trade off between the number of clock cycles needed for a program and the length of each cycle.

Many techniques that decrease the number of clock cycles may also increase the clock cycle time.

- **Do this**-A program runs in 10secs on computer A,which has a 2GHz clock. You as a designer build a Computer B which will run the same program in 6secs.As a designer you determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design,causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate will the designer has to target?

- CPU time$_A$ =CPU clock cycles$_A$ /Clock rate$_A$

  10 secs=CPU clock cycles$_A$/2×10^9 cycles/secs

CPU clock cycles$_A$= 20×10^9 cycles

CPU time$_B$=1.2 $\times$ CPU clock cycles$_A$/Clock rate$_B$

6 secs=$1.2 \times 20 \times 10^9$ cycles /Clock rate$_B$

Clock rate$_B$=4$\times$ 10^9 cycles/secs =4GHz.

To run the program in 6 secs, B must have twice the clock rate of A.

Instruction Performance

- **CPU execution time for a program**=CPU clock cycles for a program X Clock cycle time.
- CPU clock cycles= instructions for a program X Avg clock cycles per instruction
- Avg clock cycles per instruction(CPI).
- **Do this-Suppose we have two implementations of the same ISA. Computer A has a clock cycle time of 250ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500ps and a CPI of 1.2 for the same program. Which computer is faster for this program and by how much?**
- **The classic CPU performance equation**=

  CPU time=Instruction count X CPI X Clock cycle time

  =(Instruction count X CPI)/Clock rate

SPEC CPU Benchmark

- To evaluate two computer systems, a user would simply compare the execution time of the workload on the two computers.
- Workload=A set of programs run on computer that is either the actual collection of applications run by a user or constructed from real programs to approximate such a mix.
- **Benchmark:** A program selected for use in comparing computer performance
- **SPEC:**(Standard Performance Evaluation Cooperation)is an effort funded and supported by a number of computer vendors to create standard sets of benchmarks for modern computer systems.
- **In 1989, SPEC** created a benchmark set focusing on processor performance(SPEC89) which evolved through 5 generations.
- **SPEC CPU2006** (consists of a set of 12 integer benchmarks(CINT2006) and 17 floating point benchmarks(CFP2006))

- The integers benchmarks – C compiler,chess program,quantum computer simulations.
- The floating point benchmarks: Structure grid codes for finite element modeling, particle method codes for molecular dynamics,sparse linear algebra codes for fluid dynamics.
- Check for SPEC14.
- SPECRatio=$EXE\ time_{REF}$/EXE Time
- Take Geometric mean of SPECRatios
- When comparing two computers using SPEC ratios,use the GM so that it gives the same relative answer no matter what computer is used to normalize the results.If we averaged the normalized exe time values with an AM,the results would vary depending on the computer we choose as the reference.

SPECINTC2006 benchmarks running on AMD Opteron X4 model

Des	Name	Inst count $X10^9$	CPI	Clock cycle time (sec $X10^9$)	Exe time (secs)	Ref time (secs)	SPEC ratio
Interpreted String processing	Perl	2118	0.75	0.4	637	9770	15.3
GNU C compiler	Gcc	1050	1.72	0.4	724	8050	11.1

Q1:Show that the ratio of the geometric means is equal to the geometric mean of the performance ratios, and that the reference computer of SPECRatio matters not.

Assume two computers A and B and a set of SPEC ratios for each.

$$\frac{Geometric\ mean_A}{Geometric\ mean_B}=\frac{\sqrt[n]{\prod_{i=1}^{n} SPECRatio_{A_i}}}{\sqrt[n]{\prod_{i=1}^{n} SPECRatio_{B_i}}}=\sqrt[n]{\prod_{i=1}^{n}\frac{SPECRatio_{A_i}}{SPECRatio_{B_i}}}=$$

$$\sqrt[n]{\prod_{i=1}^{n}\frac{execution\ time_{B_i}}{execution\ time_{A_i}}}=\sqrt[n]{\prod_{i=1}^{n}\frac{performance_{A_i}}{performance_{B_i}}}$$

The geometric mean of the ratios is the same as the ratio of the geometric means.

Fallacies and Pitfalls

- **Pitfall:** Expecting the improvement of one aspect of a computer to increase overall performance by an amount proportional to the size of the improvement.
- This pitfall has visited designers of both h/w and s/w.
- Ex. Suppose a program runs in 100secs on a computer,with multiply operations responsible for 80secs of this time.How much do you have to improve the speed of multiplication if you want your program to run five times faster.
- The exe time of the program after making the improvement is given by the equation known as **Amdahl's Law.**
- **Exe time after improvement = (Exe time affected by improvement/Amount of improvement) +Exe time unaffected**

Exe time after improvement=80/n +(100-80)

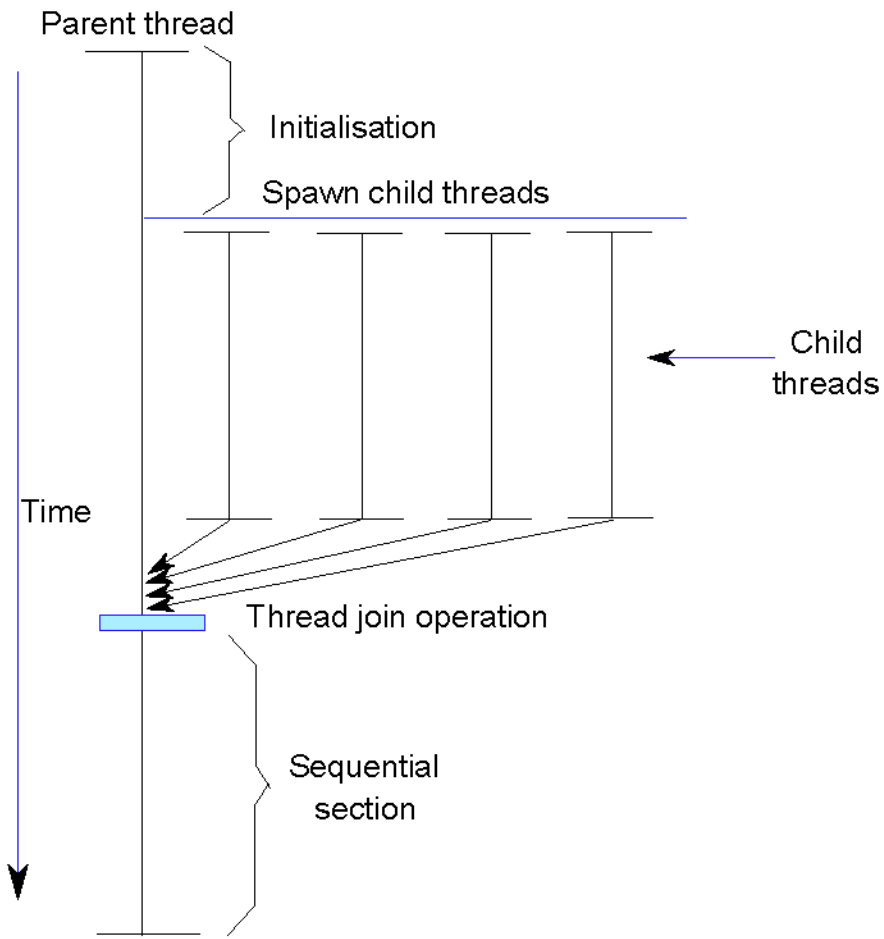
20=80/n +(100-80)

0=80/n(i.e there is no amount by which we can enhance – multiply to achieve a fivefold increase in performance ,if multiple accounts for only 80% of the workload.

The performance enhancement possible with a given improvement is limited by the amount that the improved feature is used(it's the **law of diminishing returns**).

We can use Amdahl's law to estimate performance improvements when we know the time consumed for some function and its potential speedup.

Amdahl's Law


Parent thread
Initialisation
Spawn child threads
Child threads
Time
Thread join operation
Sequential section

* For ***P*** parallel processors, we can expect a speedup of P (in the ideal case)
* Let us assume that a program takes T_{old} units of time
  * Let us divide into two parts – sequential and parallel
  * Sequential portion : $T_{old} * f_{seq}$
  * Parallel portion : $T_{old} * (1 - f_{seq})$

f_{seq} =fraction of time spend in sequential part

$(1 - f_{seq})$= fraction of time spend in parallel part

* Only, the parallel portion gets sped up P times
* The sequential portion is unaffected

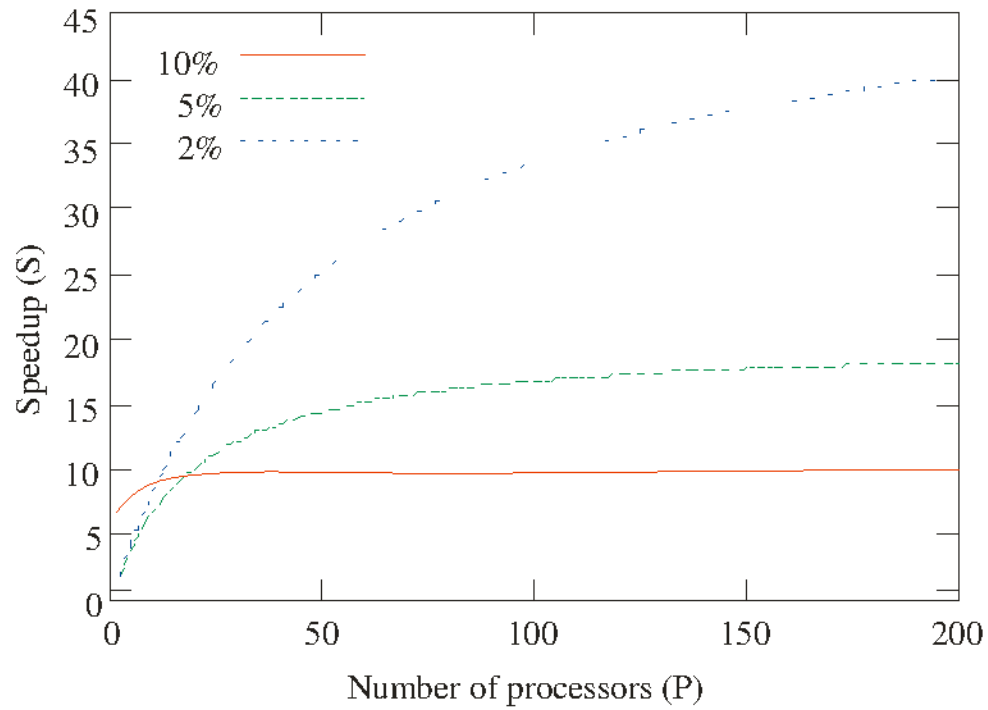
$$T_{new} = T_{old} \times \left(f_{seq} + \frac{1 - f_{seq}}{P} \right)$$

* Equation for the time taken with parallelisation
* The speedup is thus : : $\frac{T_{old}}{T_{new}}$

$$S = \frac{1}{f_{seq} + \frac{1 - f_{seq}}{P}}$$

Implications

* Consider multiple values of f_{seq}
* Speedup vs Number of processors



Conclusions

* We observe that with an increasing number of processors the speedup gradually saturates and tends to the limiting value, $1/f_{seq}$.We observe diminishing returns as we increase the number of processors beyond certain point.
* We are limited by the size of the sequential section
* For a very large number of processors, the parallel section is actually very small
* Ideally, a parallel workload should have as small a sequential section as possible.

- **Fallacy:** Computers at low utilization use little power.

Power efficiency matters at low utilizations because server workloads vary. CPU utilization for servers at Google,is between 10% to 50% most of the time.(SPEC Power Benchmark)

Server Manufacturer	Microprocessor	Total cores/sockets	Clock rate	Peak Perfm	100% load power	50% load power	10%load power	Idle power
HP	XenonE5440	8/2	3GHz	308022	269W	227W	174W	160W
Dell	XenonE5440	8/2	2.8GHz	305413	276W	230W	173W	157W
Fujitsu Seimens	XenonX3220	4/1	2.4GHz	143742	132W	110W	85W	60W

- Even servers that are only 10% utilized burn about two-third of their peak power.
- Since servers' workloads vary but use a large fraction of peak power,so we should redesign h/w to achieve "energy proportional computing".If future servers are used ,say 10% of peak power at 10% workload,we could reduce the electricity bill of datacentres(also a Concern of CO2 emissions)
- **Pitfall: Using a subset of the performance equation as a performance metric.**

Measuring performance using clock rate or CPI is a fallacy ,using two or three factors to compare performance may be valid in a limited context or misused.

Alternative to time is MIPS(Million inst per secs)=

$$Inst\ count/(Exetime \times 10^6)$$

- **Problems with MIPS:**

Cannot compare computers with different instruction sets using MIPS.

MIPS varies between programs on the same computer.(computer can not have a single MIPS rating)

If a new program executes more instructions but each instruction is faster, MIPS can vary independently from performance.

$$MIPS = \frac{IC}{\frac{IC \times CPI \times 10^6}{clock\ rate}} = \frac{clock\ rate}{CPI \times 10^6}$$